

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет “Информатика и системы управления”
Кафедра “Системы обработки информации и управления”



Дисциплина “Парадигмы и конструкции языков программирования”

Отчет по рубежному контролю №2

Выполнил:
Студент группы ИУ5-34Б
Прощаев А.П.
Преподаватель:
Гапанюк Ю.Е.

Москва 2025

Код программы

FacultyRequests.py

```

class Department():
    def __init__(self, id, name, tuitionFee, facultyId):
        self.id = id
        self.name = name
        self.tuitionFee = tuitionFee
        self.facultyId = facultyId

class Faculty():
    def __init__(self, id, name):
        self.id = id
        self.name = name

class FacultyDepartment():
    def __init__(self, facultyId, departmentId):
        self.facultyId = facultyId
        self.departmentId = departmentId

def JoinFacultyToDepartmentIfOneToManyRelationship(departments, faculties):
    one_to_many = [
        (department.name, department.tuitionFee, faculty.name)
        for faculty in faculties
        for department in departments
        if department.facultyId == faculty.id
    ]

    return one_to_many

def JoinFacultyToDepartmentIfManyToManyRelationship(departments, faculties,
facultyDepartments):
    many_to_many_temp = [
        (faculty.name, facultyDepartment.facultyId, facultyDepartment.departmentId)
        for faculty in faculties
        for facultyDepartment in facultyDepartments
        if facultyDepartment.facultyId == faculty.id
    ]

    many_to_many = [
        (department.name, department.tuitionFee, facultyName)
        for facultyName, _, departmentId in many_to_many_temp
        for department in departments
        if department.id == departmentId
    ]

    return many_to_many

def PerformFirstRequest(one_to_many):
    resB1 = list(filter(lambda row: row[0].startswith('И'), one_to_many))
    return resB1

def PerformSecondTask(faculties, one_to_many):

```

```

resA2Unsorted = []

for faculty in faculties:
    facultyDepartments = list(filter(lambda row: row[2] == faculty.name,
one_to_many))
    if facultyDepartments:
        facultyTuitionFees = [row[1] for row in facultyDepartments]
        resA2Unsorted.append((faculty.name, min(facultyTuitionFees)))

resB2 = sorted(resA2Unsorted, key=lambda row : row[1])
return resB2

def PerformThirdTask(many_to_many):
    resA3 = sorted(many_to_many, key=lambda row : row[0])
    return resA3

def main():
    faculties = [
        Faculty(1, 'Информатика и системы управления'),
        Faculty(2, 'Специальное машиностроение'),
        Faculty(3, 'Инженерный бизнес и менеджмент'),

        Faculty(11, '(Другие) Информатика и системы управления'),
        Faculty(12, '(Другое) Специальное машиностроение'),
        Faculty(13, '(Другой) Инженерный бизнес и менеджмент'),
    ]

    departments = [
        Department(1, 'ИУ5', 449000, 1),
        Department(2, 'СМ12', 539000, 2),
        Department(3, 'ИУ7', 549000, 1),
        Department(4, 'ИБМ3', 439000, 3),
        Department(5, 'СМ7', 459000, 2)
    ]

    facultyDepartments = [
        FacultyDepartment(1, 1),
        FacultyDepartment(2, 2),
        FacultyDepartment(1, 3),
        FacultyDepartment(3, 4),
        FacultyDepartment(2, 5),

        FacultyDepartment(11, 1),
        FacultyDepartment(12, 2),
        FacultyDepartment(11, 3),
        FacultyDepartment(13, 4),
        FacultyDepartment(12, 5)
    ]

    one_to_many = JoinFacultyToDepartmentIfOneToManyRelationship(departments,
faculties)
    many_to_many = JoinFacultyToDepartmentIfManyToManyRelationship(departments,
faculties, facultyDepartments)

```

```

PerformFirstRequest(one_to_many)
PerformSecondTask(faculties, one_to_many)
PerformThirdTask(many_to_many)

if __name__ == "__main__":
    main()

```

test_FacultyRequests.py

```

import unittest

from FacultyRequests import *

class TestFirstRequest(unittest.TestCase):
    def setUp(self):
        self.faculties = [
            Faculty(1, 'Информатика и системы управления'),
            Faculty(2, 'Специальное машиностроение'),
            Faculty(3, 'Инженерный бизнес и менеджмент'),

            Faculty(11, '(Другие) Информатика и системы управления'),
            Faculty(12, '(Другое) Специальное машиностроение'),
            Faculty(13, '(Другой) Инженерный бизнес и менеджмент'),
        ]

        self.departments = [
            Department(1, 'ИУ5', 449000, 1),
            Department(2, 'СМ12', 539000, 2),
            Department(3, 'ИУ7', 549000, 1),
            Department(4, 'ИБМЗ', 439000, 3),
            Department(5, 'СМ7', 459000, 2)
        ]

        self.facultyDepartments = [
            FacultyDepartment(1, 1),
            FacultyDepartment(2, 2),
            FacultyDepartment(1, 3),
            FacultyDepartment(3, 4),
            FacultyDepartment(2, 5),

            FacultyDepartment(11, 1),
            FacultyDepartment(12, 2),
            FacultyDepartment(11, 3),
            FacultyDepartment(13, 4),
            FacultyDepartment(12, 5)
        ]

        self.one_to_many =
JoinFacultyToDepartmentIfOneToManyRelationship(self.departments, self.faculties)

```

```

        self.many_to_many =
JoinFacultyToDepartmentIfManyToManyRelationship(self.departments, self.faculties,
self.facultyDepartments)

def test_PerformFirstRequest(self):
    result = PerformFirstRequest(self.one_to_many)

    correct_result = [
        ('ИУ5', 449000, 'Информатика и системы управления'),
        ('ИУ7', 549000, 'Информатика и системы управления'),
        ('ИБМЗ', 439000, 'Инженерный бизнес и менеджмент')
    ]

    self.assertEqual(result, correct_result)

def test_PerformSecondRequest(self):
    result = PerformSecondTask(self.faculties, self.one_to_many)

    correct_result = [
        ('Инженерный бизнес и менеджмент', 439000),
        ('Информатика и системы управления', 449000),
        ('Специальное машиностроение', 459000)
    ]

    self.assertEqual(result, correct_result)

def test_PerformThirdRequest(self):
    result = PerformThirdTask(self.many_to_many)

    correct_result = [
        ('ИБМЗ', 439000, 'Инженерный бизнес и менеджмент'),
        ('ИБМЗ', 439000, '(Другой) Инженерный бизнес и менеджмент'),
        ('ИУ5', 449000, 'Информатика и системы управления'),
        ('ИУ5', 449000, '(Другие) Информатика и системы управления'),
        ('ИУ7', 549000, 'Информатика и системы управления'),
        ('ИУ7', 549000, '(Другие) Информатика и системы управления'),
        ('СМ12', 539000, 'Специальное машиностроение'),
        ('СМ12', 539000, '(Другое) Специальное машиностроение'),
        ('СМ7', 459000, 'Специальное машиностроение'),
        ('СМ7', 459000, '(Другое) Специальное машиностроение')
    ]

    self.assertEqual(result, correct_result)

if __name__ == "__main__":
    unittest.main()

```

Результат работы программы

```
proshchaevalexey@MacBook-Air-Aleksej-2 rk_2 % python3 -m unittest ./test_FacultyRequests.py
...
-----
Ran 3 tests in 0.000s

OK
proshchaevalexey@MacBook-Air-Aleksej-2 rk_2 %
```